

DNS & How the Internet Really Works

Phase 1 · Fundamentals · Estimated time: 1 hour

1. Concept From First Principles

Computers address each other using IP addresses (e.g., 142.250.193.14) — not names like "swiggy.com." Humans are terrible at remembering numbers and great at remembering names, so we need a translation layer between the name a human types and the number a computer needs. That translation layer is the **Domain Name System (DNS)** — effectively the internet's phone book.

Backend engineer analogy

DNS is exactly like a config file that maps environment names to database hosts — except it's global, distributed, and cached at every layer. When you set **DB_HOST=db.internal.company.com** instead of a raw IP in your **.env** file, you're relying on the same idea: names should be resolvable to addresses, and that mapping should be changeable without touching every client.

How a DNS lookup actually resolves

DNS is organized as a hierarchy, not a single giant lookup table. When you request **api.swiggy.com**, resolution roughly follows this chain: browser cache → OS cache → router cache → ISP's recursive resolver → root DNS servers → TLD (.com) servers → Swiggy's authoritative name servers, which finally return the IP address. In practice, almost every lookup is answered by a cache long before it reaches the root servers — that caching is what makes DNS fast enough to be invisible.

One more detail that surprises engineers coming purely from application code: DNS resolution isn't free even when cached — each hop in the hierarchy that actually gets queried adds real network latency, which is why the caching at every layer (browser, OS, ISP resolver) matters so much for perceived performance. A cold DNS lookup that has to traverse the full chain can add tens to low hundreds of milliseconds before your actual application request even begins, which is one reason performance-conscious teams sometimes use DNS prefetching (resolving a domain ahead of time, before the user actually needs it).

Key DNS record types you should recognize

- **A record:** maps a domain name directly to an IPv4 address.
- **AAAA record:** same, but for IPv6.
- **CNAME record:** maps a domain name to *another* domain name (an alias), useful for pointing subdomains at a CDN or load balancer.
- **TTL (Time To Live):** how long a resolver is allowed to cache a record before re-checking — a critical lever in system design.

TTL matters a lot in real incidents. If you need to fail over to a new server fast (e.g., during an outage), a DNS record with a 24-hour TTL means some clients keep hitting the dead server for up to 24

hours even after you fix the record. Production systems that need fast failover deliberately keep DNS TTLs low (seconds to a few minutes) for exactly this reason.

It's also worth understanding DNS as an attack surface, since interviewers occasionally probe this. A **DDoS attack on a DNS provider** can take down every domain that provider resolves, even though none of those domains' actual application servers are unhealthy — this is exactly the class of incident that hit several major websites simultaneously during the 2016 Dyn DNS attack. **DNS spoofing/cache poisoning** is a separate concern where an attacker tricks a resolver into caching a malicious IP for a legitimate domain; **DNSSEC** exists specifically to cryptographically verify DNS responses and defend against this. You don't need deep DNSSEC knowledge for HLD interviews, but knowing DNS is a real attack surface — not just a lookup mechanism — signals maturity when security comes up.

2. Real-World Example / Case Study

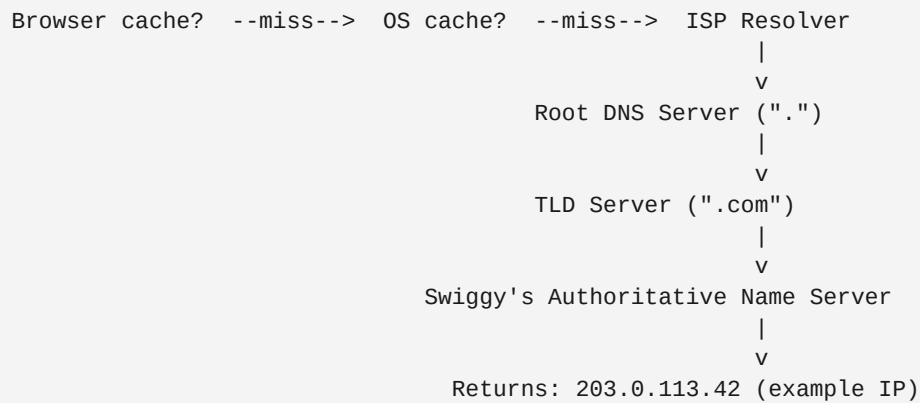
When Flipkart runs a flash sale (e.g., Big Billion Days), they often shift traffic between data centers or scale out new server pools. Part of that operational playbook involves DNS-based traffic management: routing policies (like weighted or latency-based routing, common on AWS Route 53) send a percentage of users to different regions or send all users to the nearest healthy region. This is DNS being used as a first line of load distribution, before a request even reaches a load balancer.

A very common outage pattern — worth knowing for interviews — is a "DNS outage" that looks like the whole app is down, when actually every backend server is healthy but the DNS provider itself is unreachable, so no client can even resolve the domain to begin a request. This is why some Tier-1 companies use multiple DNS providers for redundancy.

A second useful example: mobile apps (Careem, Talabat) often ship with a short list of fallback IPs or use aggressive client-side DNS caching with sensible retry logic, precisely because a phone on a flaky mobile network is far more likely to experience a failed or slow DNS lookup than a desktop browser on stable broadband. This is a deliberate resilience decision at the client layer, not just the server side — a detail candidates who've only worked on web backends sometimes miss when asked to design mobile-first products for the Gulf or Indian markets, where network conditions vary widely by region.

3. Diagram

User types: api.swiggy.com



4. Trade-offs Table

Decision	Low TTL (e.g., 30s)	High TTL (e.g., 24h)
Failover speed	Fast — changes propagate in seconds	Slow — stale entries persist for hours
DNS server load	Higher — resolvers re-query often	Lower — fewer repeat lookups
Best for	Systems needing fast failover / frequent IP changes	Stable, rarely-changing infrastructure

5. Common Interview Questions

Q1: "What happens if DNS resolution fails?"

Model approach: Explain that the client can never even open a TCP connection, so the app appears fully down to users, even if every backend server is healthy. Mention mitigation: multiple DNS providers, client-side DNS caching with fallback IPs for critical infra.

Q2: "How would you use DNS to route users to the nearest data center?"

Model approach: Describe latency-based or geo-based DNS routing (e.g., AWS Route 53, Cloudflare) where the DNS provider returns a different IP depending on the requester's location, sending Gulf-region users to a UAE data center and Indian users to a Mumbai data center.

Q3: "How would you use multiple DNS providers for redundancy, and what problem does that solve?"

Model approach: Explain that relying on a single DNS provider makes that provider a single point of failure for your entire domain — if it goes down, no client can resolve your domain at all, regardless of your application's health. Running two independent providers (each holding the authoritative records) removes that single point of failure, at the cost of extra operational complexity keeping records in sync between them.

6. Practice Exercises

- 1 Explain, in your own words, why lowering DNS TTL before a planned migration is a common ops practice.
- 2 List two DNS record types and one real scenario where each would be used in a system you've worked on.
- 3 Describe what a user experiences if your app's DNS provider goes down but every application server is fine.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"DNS Explained: The System That Makes the Internet Work"	Clear, up-to-date overview of the full DNS chain.
English	"How DNS Works" — Computerphile	Well-respected CS channel, goes slightly deeper technically.
Hindi	"DNS Kya Hota Hai? DNS Explained in Hindi"	Beginner-friendly Hindi walkthrough of DNS basics.
Hindi	"What is DNS & How It Works? (DNS Explained in Hindi)" — zeerothink	Covers record types and resolution flow in Hindi.

8. Key Takeaways

- DNS translates human-readable domain names into IP addresses computers can route to.
- Resolution follows a hierarchy: browser/OS cache → ISP resolver → root → TLD → authoritative server.
- TTL controls how long a DNS answer is cached — a key lever for failover speed vs. DNS load.
- DNS can be used deliberately for traffic routing (geo/latency-based) before requests even reach your load balancer.
- A DNS outage can make a fully healthy backend look completely down to users.
- Production-grade systems often use multiple DNS providers for redundancy.